

n-gram

November 23, 2025

```
[1]: %reload_ext autoreload
      %autoreload 2
```

```
[2]: # from kret_studies import *
      from kret_studies.notebook import *

      # from kret_studies.complex import *

      # logger = get_notebook_logger()
```

Loaded environment variables from /Users/Akseldkw/coding/Columbia/UML-Project/.env.

INFO:datasets:JAX version 0.7.2 available.

```
[4]: from uml_project import *

      HF_DIR, HF_REGISTRY, MODEL_DIR, DATA_DIR
```

```
[4]: (PosixPath('/Users/Akseldkw/coding/Columbia/UML-Project/data/huggingface'),
      PosixPath('/Users/Akseldkw/coding/Columbia/UML-Project/data/huggingface/REGISTRY.json'),
      PosixPath('/Users/Akseldkw/coding/Columbia/UML-Project/models'),
      PosixPath('/Users/Akseldkw/coding/Columbia/UML-Project/data'))
```

```
[5]: # ! pip install kenlm
```

0.1 Download model

```
[6]: from huggingface_hub import hf_hub_download
      import kenlm

      model_path = hf_hub_download(
          repo_id="edugp/kenlm",
          filename="wikipedia/en.arpa.bin", # exact name from repo listing
      )
      lm = kenlm.LanguageModel(model_path)
```

```
[15]: lm, model_path
```

```
[15]: (<Model from b'en.arpa.bin'>,
      '/Users/Akseldkw/.cache/huggingface/hub/models--edugp--
      kenlm/snapshots/3fbc35c83b1a39f420a345b7c96a186c8030d834/wikipedia/en.arpa.bin')
```

0.2 Model Sandbox/Testing

```
[ ]: def sentence_logprob(model: kenlm.LanguageModel, sentence: str) -> float:
      return model.score(sentence, bos=True, eos=True)

def sentence_perplexity(model: kenlm.LanguageModel, sentence: str) -> float:
    words = sentence.split()
    log_prob = sentence_logprob(model, sentence)
    avg_log_prob = log_prob / len(words)
    return 10 ** (-avg_log_prob)
```

```
[14]: logprob, perplexity = sentence_logprob(lm, "Hello world"),
      ↪sentence_perplexity(lm, "Hello world")
      print(f"Log-probability: {logprob:.2f}, Perplexity: {perplexity:.2f}")
```

Log-probability: -16.94, Perplexity: 294,304,383.04

```
[11]: sentence_perplexity(lm, "This is a test sentence.")
```

```
[11]: 1496175.2021246448
```

0.3 Model Eval Functions

```
[ ]: import math

def sentence_stats(model: kenlm.LanguageModel, sentence: str) -> tuple[float,
↪float]:
    """
    Combine log-probability and perplexity calculation.

    Returns (avg_nll, ppl) where:
        avg_nll = per-token negative log10 probability (lower is better)
        ppl    = 10 ** avg_nll (standard base-10 perplexity)
    """
    tokens = sentence.split()
    if not tokens:
        return float("inf"), float("inf")

    log10_p = model.score(sentence, bos=True, eos=True) # log10 P(sentence)
    avg_nll = -log10_p / len(tokens) # per-token NLL in log10
    ppl = 10**avg_nll
    return avg_nll, ppl
```

```
[20]: logprob, perplexity = sentence_stats(lm, "Hello world")
print(f"Avg NLL: {logprob:,.2f}, Perplexity: {perplexity:,.2f}")
```

Avg NLL: 8.47, Perplexity: 294,304,383.04

```
[ ]: def doc_similarity_score(lm: kenlm.LanguageModel, s1: str, s2: str) -> float:
    # symmetric "compatibility" based on cross-perplexity
    avg_nll_1, _ = sentence_stats(lm, s1)
    avg_nll_2, _ = sentence_stats(lm, s2)
    # also consider concatenation to capture cross-context
    concat = s1 + " " + s2
    avg_nll_concat, _ = sentence_stats(lm, concat)
    # lower means more compatible; flip sign for higher-is-better
    return -(avg_nll_1 + avg_nll_2 + avg_nll_concat) / 3.0
```

0.4 Embed to sentence vector

```
[ ]: def sentence_features(lm: kenlm.LanguageModel, sent: str) -> np.ndarray:
    tokens = sent.split()
    if not tokens:
        return np.zeros(4)

    # full_scores gives per-token contributions
    log10_scores = []
    for _, _, _, log10_p in lm.full_scores(sent, bos=True, eos=True):
        log10_scores.append(log10_p)
    log10_scores = np.array(log10_scores)

    nll = -log10_scores # per-token negative log10 prob
    return np.array([
        nll.mean(),
        nll.std(),
        nll.max(),
        nll.min(),
    ])
)
```

```
[ ]: def avg_nll(lm: kenlm.LanguageModel, sent: str) -> float:
    tokens = sent.split()
    if not tokens:
        return 0.0
    lp = lm.score(sent, bos=True, eos=True) # log10 P
    return -lp / len(tokens)

def mixture_lm_embed(lms: list[kenlm.LanguageModel], sent: str) -> np.ndarray:
    return np.array([avg_nll(lm, sent) for lm in lms], dtype=float)
```